

Thesis Report

BLITZKRIEG Studio

By Yloan, Quentin, Aegon, David & Adam

Table of Contents

1- Game presentation & principles.....	2
The idea.....	2
The originality.....	3
The chara-design development.....	4
Character presentation.....	4
Fire — The All-Rounder.....	5
Water — The Zoner.....	7
Electric — The Setup Fighter.....	10
Ghost — The Glass Cannon.....	13
Food — The Debuffer.....	17
Map Presentation.....	21
Map 01.....	22
Map 02.....	23
Map 03.....	24
The Crystal Cave.....	24
Mysterious · Luminous · Otherworldly.....	24
Inspiration.....	25
2- Technical presentation.....	26
OOP.....	26
Multiplayer engine.....	27
AI.....	28
Attack system.....	29
Character 1 (Fire).....	29
Character 2 (Water).....	30
Character 3 (Ghost).....	30
Character 4 (Electric).....	31
Character 5 (Food).....	32
Menu.....	32
Session's system.....	32
Maps selection.....	33
Character selection.....	34
Gameplay.....	34
Art.....	34
Characters.....	35
Menus.....	36
Maps.....	36
3- Issues encountered during the development.....	36
4- The website.....	38

5- Each group member's implication.....	39
Yloan.....	39
David.....	40
Adam.....	41
QUENTIN.....	43
AEGON.....	44
6- Communication and organization.....	46
Team Organisation.....	46
Communication Tools - Discord.....	47
Regular Meetings.....	49
Summary.....	51
7- IF we can restart, what will we change.....	52
8- Conclusion (what was learnt by the end of the project).....	53
Controls.....	57

1- Game presentation & principles

The idea

The idea was to make a game inspired by pokemon and double dragon, which is a mix between the principles of street fighter (battle of characters inspired by pokemon and robot) and a battle royale's gameplay of at least 4 players of each 3 character (1 main character and 2 supports).

Blitzkrieg is a fast paced multiplayer team based fighting game where you can play as a large cast of robots with various unique designs, play styles, and synergies.

In the game, up to four people can play against each other using a vast array of characters.

Before a game begins, each player chooses three of the 5 playable robots before duking it out on one of the 3 available maps.

The last one standing is declared the winner.

The game put a great emphasis on team-building, strategy, and fast thinking.

The originality

There is a unique way that Blitzkrieg stands out from traditional fighting games through its use of status and synergy systems. Instead of focusing on either hit points or direct damage dealt, most of the attacks in this game give off one or more of the following 8 unique statuses (Burn, Wet, Oiled, Pushed, Weakened, Bleed, Opened Paralyzed), where each status has its own functionality. The complexity of the system lies in how different status effects interact with each other – An oil slicked foe being set alight will create a large explosion; pushed while wet, an opponent will go further than normal; an opponent who is weakened due to being dealt damage through a slashing attack will begin to bleed (this effect will begin immediately). These types of interactions give rise to another level of emergent strategy as you align your attacks in a specific sequence and create a balanced team. Generally, the type of emergent strategy that Blitzkreig

brings to the game does not exist within the other arcade-type fighting games of equal value through their respective components.

The game's support mechanism has added even more depth and uniqueness to the overall experience than just providing players with different levels of status. Players who aren't in the game as active participants can use one of their support skills (which includes cross-character combos) while they wait off the field in a variety of ways that reward players for making careful selections of their rosters. In addition to featuring 6 different playable characters, each of whom has their own stats, resistances, and attack types, the game also features six different maps, three of which have environmental hazards on them. Blitzkrieg has tremendous potential for being replayed, as well as providing players with great tactical options.

The chara-design development

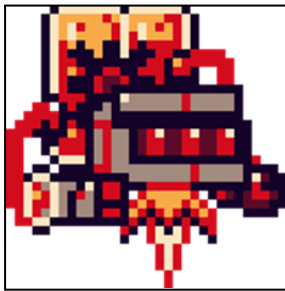
The different types of characters are created with different themes like fire, water, electricity, poison, ghost, food, etc., and the pixel art identity (the graphic style of the character) is created using Aseprite, which has also allowed us to do all the animation of each character. Each character has a complete set of animations that contain different animations for standing still, moving, hurting, dying, and applying their skills (with the visual effects such as electric shock, fireballs, explosions, slashes, etc.). A fully animated character has about 75 total frames, which are used in all of the animations. The character art makes up the highest percentage of the total time spent on the project.

The character design is also created with consideration for the gameplay. The first characters were created in a way that allowed the design to create all the requirements necessary for the game to function properly, and that would make it possible for 11 out of the game's 15 requirements to be developed early enough in the development process to allow for the production of both art and gameplay to progress simultaneously without creating any production delays.

Character presentation

This section presents the playable and non-playable characters developed for the game, detailing their design intentions, gameplay roles, and key attributes. Each entry combines a report-style overview suitable for design documentation with a structured character sheet summarising their core statistics and abilities.

Fire — The All-Rounder



Fire

Class: All-Rounder

Element: Fire / Oil

Archetype: Balanced Fighter with Projectile

Difficulty: ★★☆☆☆ (Beginner-Friendly)

Fire fills the all-rounder slot in the game's character roster. This means their stats are intentionally balanced across offence, defence, and mobility; no single attribute is exceptional, but none is a liability either. This makes Fire an ideal character for new players learning the game, while still offering enough depth for experienced players to master through precise use of their unique fire-and-oil combo mechanic.

Their core gameplay loop revolves around two complementary elements:

- Fire damage — delivered through direct attacks and a signature fireball projectile, dealing consistent damage at range and applying a short burn effect on hit.
- Oil application — a secondary mechanic that coats enemies in flammable oil, dramatically amplifying the damage of any subsequent fire-based attack. Combining oil and fire in the right sequence is the key to maximising Fire's damage output and is what separates an average Fire player from a skilled one.

This synergy between oil and fire creates a high-reward combo system that is easy to understand but satisfying to execute, embodying the design goal of making Fire accessible without making it shallow.

BASE STATISTICS	
Attack	 7/10
Durability	 6/10
Speed	 6/10

ABILITY	DESCRIPTION	TYPE
Fireball	Launches a mid-speed fire projectile. Applies a burn effect on hit, dealing damage over time.	Projectile / Ranged

Oil Slick	Coats a target or area in flammable oil. Does no damage alone, but massively amplifies fire damage when combined.	Utility / Debuff
Ignite Combo	Signature combo: apply Oil Slick first, then trigger with any fire attack for a devastating burst of damage. Core mechanic for maximising output.	Combo / Special

Water — The Zoner



Water

Class: Zoner

Element: Water / Ice

Archetype: Long-Range Control / Crowd Disabler

Difficulty: ★★★★★☆ (High skill floor)

Water fills the zoner archetype, a character archetype defined by the ability to dictate the pace of a fight by controlling the space between combatants. Zoners thrive when they can maintain distance and punish opponents who try to approach, but struggle significantly when that distance collapses. Water embodies this archetype fully: below-average durability, medium speed, but a toolkit designed to prevent opponents from ever getting comfortable.

The defining mechanic of Water's kit is the Wet status. Two of Water's three skills apply this condition, and it acts as the foundation of the character's damage identity. An opponent who is Wet takes amplified damage from follow-up water attacks, creating a "tag and punish" gameplay loop that rewards careful sequencing. Combined with the push effect on S2 and the hard disable on S3, Water can dictate exactly where opponents are allowed to be on the field.

Water's three skills each fill a distinct tactical role:

- Tidal Lance (S1) — the primary poke and Wet applicator. Fires a pressurised jet that strikes a single target twice at long range. Used to apply Wet and maintain pressure from a safe distance.
- Tidal Surge (S2) — the emergency panic button and zone-clear tool. Releases a burst of water in all directions, dealing blunt damage to every nearby enemy and sending them flying back. Most effective when an opponent has closed the gap and close-quarters becomes unavoidable.
- Bubble Trap (S3) — the signature high-risk, high-reward ability. Water's tongue snares a distant target; if it connects, the enemy is chewed and spat out into a bubble that drifts forward for five seconds while they are fully disabled. During the spit phase (S3-2), Water itself becomes invulnerable, making a successful grab both a powerful disable and a guaranteed safe window.

BASE STATISTICS	
Attack	 7/10
Durability	 3/10

Speed

 5/10

ABILITY	DESCRIPTION	TYPE
S1 Tidal Lance	Water fires a sharp, pressurised jet that pierces through the air and strikes with two rapid bursts, soaking the first target it reaches. A precise, long-range poke that keeps enemies at arm's length and sets up the Wet combo.	Projectile / Ranged
S2 Tidal Surge	Water releases a violent burst of water in all directions, hammering every nearby enemy with blunt force and sending them flying backwards. The emergency tool of Water's kit, used to reclaim space when the distance collapses, and close-quarters becomes unavoidable.	Utility
S3 Bubble Trap	Water lashes out its tongue to snare a distant target (S3-1). On a successful grab, it chews the enemy up and spits them out encased in a bubble (S3-2), leaving them soaked, helpless, and drifting forward for 5 seconds. Water is fully invulnerable during the spit	Special

	phase — a guaranteed safe window that rewards the precision required to land the grab.	
--	--	--

Electric — The Setup Fighter



Electric

Class: Setup Fighter

Element: Electricity

Archetype: Resource Manager / Burst Damage

Mechanic: Charge System (0–4 charges, HUD battery icon)

Difficulty: ★★★★★☆ (High skill ceiling)

Electric fills the setup fighter archetype, a character who builds toward a powerful payoff rather than dealing consistent damage from the start. The central mechanic is the Charge system: Electric can hold up to 4 charges at any time, displayed via a battery icon on the HUD. Charges are gained through combat and passively over time, and spent on the most powerful abilities. Managing this resource, knowing when to build, when to hold, and when to spend, is the defining skill of playing Electric well.

The three skills each interact with the charge system differently:

- Arc Strike (S1) — the primary charge generator and damage tool. A two-phase attack that fires a projectile (S1-1); if it connects, a follow-up shock triggers (S1-2) before the animation resolves (S1-3). A hit awards 2 charges, the fastest way to build charges in the kit. A miss still plays S1-3 but yields nothing, punishing careless use.

- **Static Field (S2)** — a flexible utility skill that doubles as both a charge builder and a charge spender depending on the situation. Below 3 charges, S2-1 fires a paralyzing field that gains 1 charge. At 3 or more charges, S2-2 activates a more powerful discharge that costs 2 charges, dealing more damage and stronger paralysis in exchange for the resource investment.
- **Overload (S3)** — the signature all-in finisher. Electric expends every charge they have in a single catastrophic release of energy. The more charges stored when S3 is triggered, the more devastating the outcome. Used to close out a fight or punish an opponent who has been successfully paralyzed and repositioned.

The paralysis mechanic that ties all three skills together serves a dual purpose: it gives Electric time to reposition safely while charges build, and it sets up the optimal conditions for an Overload. A fully paralyzed opponent who cannot react is the ideal target for a full-charge S3.

BASE STATISTICS

Attack	 9/10
Durability	 5/10
Speed	 6/10

CHARGE SYSTEM

SOURCE	CHANGE	CONDITION
Passive (time)	+1 charge	Every 15 seconds
Arc Strike (S1)	+2 charges	On hit only (S1-2 triggers)
Static Field S2-1	+1 charge	When current charges < 3
Static Field S2-2	-2 charges	When current charges $\geq$ 3 (uses S2-2)
Overload (S3)	-ALL charges	Spends every stored charge on activation

ABILITY	DESCRIPTION	TYPE
S1 Arc Strike	Electric fires a charged bolt (S1-1). On hit, a follow-up shock discharges (S1-2), stunning the target briefly. Awards 2 charges on a successful hit, the fastest charge gain in the kit. Missing skips S1-2 and yields no charges.	Debuff / Ranged
S2	Behaviour depends on current charges. Below 3 charges: S2-1	Utility

Static Field	releases a grounding field that paralyses nearby enemies and generates 1 charge. At 3+ charges: S2-2 unleashes a more powerful surge, hitting harder and pushing enemies back, at the cost of 2 charges.	
S3 Overload	Electric releases every stored charge simultaneously in a single catastrophic burst. Damage scales directly with the number of charges spent; a full 4-charge Overload is the highest damage output available in the kit. Leaves Electric at zero charges and fully exposed after use.	AoE / Special

Ghost — The Glass Cannon



Ghost

Class: Glass Cannon / Debuffer

Element: Spectral / Shadow

Archetype: High-Risk Melee Assassin

Mechanic: Weakened, branches into Bleed, Paralysis, or Opening

Difficulty: ★★★★★ (Expert — punishing if misplayed)

Ghost fills the glass cannon archetype in its purest form: extreme offensive output paired with near-zero defensive options. High speed partially compensates for low durability. Ghost can often avoid hits through positioning and aggression, but any sustained engagement that does not end quickly is likely to end badly. The entire kit is built around ending fights fast.

The defining mechanic is the Weakened status, applied by Ghost's S1 and acting as the prerequisite for S2's branching options. Weakened on its own does minimal direct damage, but it opens the door to three distinct follow-up states that Ghost's S2 can inflict depending on the situation:

- Bleed — deals damage over time, punishing opponents who try to disengage and wait out a fight. Ideal against high-health targets.
- Paralysis — temporarily locks the opponent in place, creating a guaranteed window for repositioning or a follow-up attack. Ideal against fast or evasive opponents.
- Opening — marks the opponent to take x1.5 damage from all of Ghost's subsequent attacks for a duration, setting up the S3 finisher for devastating effect.

This branching structure means that S2 is never wasted; the "right" follow-up always depends on the current state of the fight, rewarding adaptable players who read the situation rather than following a fixed combo. The three options also make Ghost unpredictable: opponents who know Ghost's kit cannot be sure which debuff is coming, making defence difficult.

The three skills flow naturally from this identity:

- Phantom Slash (S1) — a three-phase sword sequence culminating in a cross-slash energy burst that applies Weakened. The core opener and the gateway to Ghost's entire damage toolkit.
- Wraith's Judgment (S2) — a charge-up dash that fires a spectral projectile. If the target is Weakened, the hit branches into one of the three debuff states. The choice of which debuff to apply is the central decision-making moment in Ghost's gameplay loop.
- Revenant's Ruin (S3) — a 36-frame full-commitment finisher. Ghost winds up into a sweeping red energy arc, then unleashes a barrage of spectral slashes and energy blades. The highest single-combo damage output in the kit, and significantly amplified if Opening is active on the target.

BASE STATISTICS	
Attack	10/10
Durability	2/10
Speed	9/10

WEAKENED — DEBUFF BRANCHES

STATE	TRIGGER	EFFECT
Weakened	Applied by S1 (Phantom Slash)	Base debuff state. Minor effect alone, but prerequisite for all S2 branches. Signals that Ghost has cracked the opponent's defences.
Bleed	S2 on Weakened target — Bleed branch	Deals damage over time. Best against high-health targets who would otherwise outlast Ghost's burst window.
Paralysis	S2 on Weakened target — Paralysis branch	Locks the opponent in place for a short duration, creating a guaranteed safe window for repositioning or follow-up, best against fast, mobile opponents.
Opening	S2 on Weakened target — Opening branch	Marks the target to take x1.5 damage from all of Ghost's attacks for a duration. The direct setup for S3. Most powerful branch in terms of raw damage ceiling, but wasted if S3 cannot be landed in the window.

ABILITY	DESCRIPTION	TYPE
S1 Phantom Slash	A three-phase sword combo: a quick wind-up (S1-1), followed by a sweeping cross-slash that releases a burst of spectral energy on contact (S1-2), and a follow-through recovery (S1-3). Applies Weakened on hit. The gateway to Ghost's entire debuff toolkit and the required opener before S2 can branch.	Debuff / Melee
S2 Wraith's Judgment	Ghost charges briefly before dashing forward and firing a spectral projectile. On a standard hit, deals moderate damage. If the target is Weakened, the hit branches into one of three debuff states: Bleed (damage over time), Paralysis (movement lock), or Opening (x1.5 damage vulnerability). The branch chosen reflects the current tactical need.	Melee / Debuff
S3 Revenant's Ruin	Ghost's full-commitment finisher. A 36-frame sequence: sword drawn back, a sweeping crimson energy arc, then a relentless barrage of spectral slashes and energy blades. Deals the highest combo damage in the kit. Damage is massively amplified if	AoE / Special

	Opening is active on the target. High animation commitment, Ghost is fully vulnerable if S3 is launched without a clear opening.	
--	--	--

Food — The Debuffer



Food

Class: Debuffer / Area Denier

Element: Water / Oil

Archetype: Slow Tank with Status Control

Mechanic: Can System — 4 Water cans + 4 Soda cans (HUD)

Difficulty: ★★☆☆☆ (Moderate — position-dependent)

Food fills a unique niche in the roster: a slow, high-health frontliner whose primary output is not damage but status disruption. The two status effects at the heart of the kit, Slowed (applied by oil) and Slippery (applied by water), make enemies erratic and difficult to control for the opposing player, effectively reversing the usual frustration dynamic of a slow character. Food cannot be outrun. The opponent can be made to struggle to run at all.

The Can System is Food's central resource mechanic: four water cans and four soda cans are tracked on the HUD. Each drink skill consumes one can of the corresponding type, and when both are depleted, Food is forced to use S2, a blunt close-range shove that is dangerous only because the opponent has to get near a high-health tank to receive it. Reloading with S3 restores both can types,

but requires committing to an animation window, creating a genuine resource management loop.

The three skills each serve a distinct role:

- Hydro Toss (Water Can — Skill 1) — Food hurls a water can at a target, soaking them on impact. The water makes the opponent's movement unpredictable and slippery, causing them to slide uncontrollably and making precision movement nearly impossible for the opponent. Consumes 1 water can.
- Grease Splash (Soda Can — Skill 2) — Food launches a soda can that bursts into an oil slick on contact, coating the target in viscous grease. Oiled enemies move at reduced speed, making it significantly harder for them to disengage or dodge. Consumes 1 soda can.
- Last Resort Shove (S2 — no cans) — When Food has no cans of the relevant type remaining, the first and second skills are replaced by a blunt close-range shove. It deals modest damage and pushes the target back, but requires Food to be in close quarters, a dangerous position that is also exactly where Food's high health makes it most threatening.
- Restock (S3) — Food triggers a full reload, replenishing all water and soda cans back to maximum. Commits Food to an animation window during which it is vulnerable, creating a risk-reward decision: reload early and stay stocked, or wait until empty and risk being caught in the transition.

BASE STATISTICS	
Attack	5/10
Durability	8/10

Speed	2/10
-------	-----------------------------

CAN SYSTEM			
CAN TYPE	MAX STOCK	USED BY	OUT OF AMMO
Water Can 💧	4	Skill 1 (Hydro Toss)	Skill 1 → S2 (Last Resort Shove)
Soda Can 🥤	4	Skill 2 (Grease Splash)	Skill 2 → S2 (Last Resort Shove)
Reload (S3)	—	Restocks all cans to max 4+4	Vulnerable during reload animation

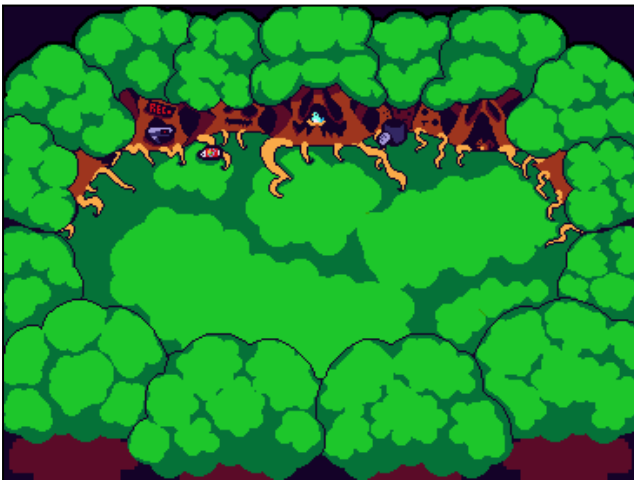
ABILITY	DESCRIPTION	TYPE
Skill 1 Hydro Toss	Food hurls a water can at a target. On impact, the soaking causes the enemy's movement to become slippery and erratic, and the opponent loses precise directional control, making it difficult to commit to	Debuff / Ranged

	deliberate movement. Consumes 1 water can. Becomes S2 if no water cans remain.	
Skill 2 Grease Splash	Food launches a soda can that bursts into an oil slick on contact, coating the target in sticky grease. Oiled enemies are significantly slowed, reducing their movement speed and making disengagement difficult. Consumes 1 soda can. Becomes S2 if no soda cans remain.	Debuff / Ranged
S2 Last Resort Shove	When the relevant can type is depleted, Food falls back on a close-range blunt shove. It deals modest damage and pushes the target back, but requires Food to be in close quarters, a risky position offset by Food's considerable health advantage. No can consumed.	Melee
S3 Restock	Food triggers a full reload, replenishing all water and soda cans to maximum (4+4). Commits Food to a vulnerability window during the animation. Timing this reload carefully, preferably when opponents have been debuffed and are struggling to close in, is a key pillar of playing Food effectively.	Special

Map Presentation

Blitzkrieg features three distinct maps, each with its own visual identity, atmosphere, and design intent. All maps share the same arena structure — a central open combat zone bordered by environmental elements — but differ significantly in tone and aesthetic. Each map was designed to complement a different emotional register of the game, from the dense and organic to the arid and hostile, and the mysterious and otherworldly. Together they provide variety to the player experience while maintaining visual coherence with the game's overall pixel art style.

Map 01



The Forest

Dense · Organic · Claustrophobic

The Forest is the game's most enclosed arena. Thick canopies of deep green foliage press in from all sides, framing a central clearing that serves as the primary combat zone. Gnarled roots and twisted branches line the borders, and the colour palette, rich greens, earthy browns, and warm amber, gives the map a sense of overgrown, untamed wilderness. The purple glowing eyes peering from within the treeline add an unsettling edge, hinting that the forest is not entirely lifeless.

- Enclosed layout encourages close-range and mid-range combat.

- Dense foliage border clearly delineates the play area.
- Dark, organic aesthetic contrasts sharply with the Desert and Crystal Cave.
- Pixel art style aligns closely with the game's retro visual identity.

Map 02



The Desert

Open · Harsh · Sunbaked

The Desert is the game's most open and exposed arena. Rolling dunes of warm terracotta stretch across the background beneath a bright sky dotted with sparse clouds and a blazing pixel sun. Scattered cacti and small bone-like debris dot the landscape, reinforcing the hostility of the environment. The central combat area is a wide, flattened expanse that gives players significant room to manoeuvre, making it the most movement-dependent of the three maps and favouring ranged and zoning playstyles.

- Open layout rewards ranged characters and movement-based strategies.
- Bright, warm colour palette creates a strong visual contrast to the other two maps.
- Environmental details, cacti, bones, blazing sun, reinforce the hostile atmosphere.
- Wide play area gives players more room to disengage and reposition.

Map 03



The Crystal Cave

Mysterious · Luminous · Otherworldly

The Crystal Cave is the most visually striking of the three maps and the one that departs most dramatically from the pixel art aesthetic of its counterparts. A sprawling underground cavern, lit by glowing blue waterfalls, luminous purple crystals, and bioluminescent flora, the map creates an atmosphere of eerie beauty. A stone staircase at the far end leads to a darkened archway, suggesting a place of power and hidden danger. The cracked stone floor of the

central arena and the richly detailed borders make this the most elaborate environment in the game.

- Distinct visual style, higher detail and richer colour depth than the other maps.
- Central arena framed by glowing crystals and waterfalls creates a dramatic backdrop.
- Dark, cool colour palette (blues, purples) provides strong contrast to the Forest and Desert.
- Atmospheric design reinforces the game's tonal range and world-building ambition.

The three maps were designed to offer players meaningfully different visual and spatial experiences while keeping the core gameplay structure consistent. The decision to vary the aesthetic range deliberately, from the organic closeness of the Forest to the open hostility of the Desert to the luminous grandeur of the Crystal Cave, was driven by a desire to give the game visual depth and replayability beyond the character roster alone.

The Forest and Desert maps were created entirely by the team's graphic designer using Aseprite, consistent with the pixel art style established across the rest of the game. The Crystal Cave, by contrast, adopts a higher-fidelity visual style, reflecting a deliberate creative choice to give the third map a more dramatic and distinct identity. All three maps were designed with the same arena layout logic, a clear, open central combat zone bordered by environmental elements, ensuring that the gameplay experience remains fair and readable regardless of which map is selected.

Inspiration

The idea behind the project has its roots in a personal passion that one of our team members, our graphic designer, had carried since childhood. Years before this project came to be, he had taken part in a drawing competition centred around designing mechanical, robot-like creatures, imaginative characters that blended organic shapes with industrial, mech-inspired aesthetics. That early creative experience left a lasting impression, and the distinctive visual style he developed during that period became the seed from which this project ultimately grew.

When the team came together and began discussing what kind of game we wanted to make, he brought that vision to the table. The idea of a fast-paced, character-driven fighting game built around a roster of uniquely designed combatants gave the whole team something to rally around, and it quickly became clear that this was the direction we wanted to pursue. Each character in the roster, from the explosive aggression of Fire to the methodical control of Water, the calculated power of Electric, and the spectral precision of Ghost — can be traced back in spirit to that original impulse to create creatures that feel both mechanical and alive.

Beyond this personal origin, the project drew inspiration from a number of established titles that the team admired. The old-school pixelated art style was heavily influenced by classic and modern indie games that embrace retro aesthetics, and the character-based combat system drew loose parallels to iconic fighting games where each character offers a meaningfully different playstyle. The music direction followed a similar logic, with games like *Nuclear Throne* serving as a reference point for the kind of atmosphere we wanted the soundtrack to evoke.

What made this project meaningful beyond its technical scope was that it started as a genuine creative idea rather than an academic exercise. That origin gave the whole team a sense of ownership and investment in the final product that shaped every decision made along the way.

2- Technical presentation

OOP

Object-Oriented Programming (OOP) is a very useful paradigm for making games.

In OOP, a Class is like a mold or a blueprint, while an Object is the actual thing created from that mold.

Every object has two main characteristics:

- Attributes (Data): What the object *knows* or *has* (e.g., a character's health, speed, or name).
- Methods (Functions): What the object *does* or *can do* (e.g., jump, attack, or take damage).

For example, you can create a single *Monster* class (the mold). Then, you can use it to instantly create 100 individual monster objects in your game, each with its own health pool and position, but all sharing the same basic behavior.

In the actual game, there are many objects: *Character*, *Bot*, *Game*, *Menu*, and so on.

We also use *subclasses* that inherit methods and attributes from a main class. For example, specific characters like *Water* or *Furnace* inherit from the base *Character* class. They reuse its core logic but *override* certain methods to give each character its own unique behavior.

Multiplayer engine

The multiplayer engine is a critical component of the game, as the entire gameplay experience takes place online. The server runs on a machine with a public IP address and is hosted using *Katabump*. While *Katabump* is a great free

solution, it requires the server to be manually renewed every four days, with no current possibility of automating this process.

The server architecture operates as follows:

- **Initialization:** When the server object is initialized, a dedicated listener thread is started to constantly listen for incoming client connection requests.
- **Client Connection:** As soon as a client connects, the server spawns a new handler thread dedicated exclusively to managing that specific client's messages.
- **Data Management:** The client is then appended to the active clients list, which is stored as an attribute within the main *Game* object.

Once this setup is complete, the server is fully operational, continuously receiving, processing, and broadcasting network messages to keep the game synchronized.

AI

A dictionary that associates action names with their corresponding execution functions is used by the AI, which is implemented through a Bot object, to manage its possible moves. Flee, assault, follow the nearest player, or perform a random action (move top, bottom, left, right, or just rest) are the main options.

Behavioral Modes: The bot constantly cycles between Offensive and Defensive modes. These modes last for a randomized duration and completely alter the probability pool of the bot's decisions.

- ***Offensive Mode:*** Highly aggressive, heavily prioritizing following and attacking the player.
- ***Defensive Mode:*** Cautious behavior, prioritizing fleeing, resting, and maintaining distance, only attacking under safer conditions.

The bot monitors incoming damage through the "Rage" Mechanic. It overrides its current state and is compelled to enter Offensive mode for a prolonged period of time if it sustains a substantial amount of damage within a brief time window (RAGE_WINDOW_FRAMES).

Health and Distance Thresholds: The bot's active mode, distance from the target, and remaining health all have a significant impact on the action selection process.

Health $\leq$ 20%: The most important thing is to survive. Although aggressive mode may rarely cause a frantic attack or follow, the bot will typically opt to run away.

A tactical state where the bot determines whether the player is within skill range is $20\% < \text{Health} \leq 60\%$. It will balance assaulting, following, or tactical retreats based on the active mode.

Health $> 60\%$: The bot behaves with assurance. While defensive mode keeps a structured balance of moving, following, and attacking when it's most effective, offensive mode initiates intense chase and attacks.

Smart Skill Positioning: Bots don't merely approach players head-on. Before performing a skill, they firmly align themselves on the Y-axis (Y_ALIGNMENT_TOLERANCE) and use character-specific data to determine the ideal horizontal distance. Additionally, characters 4 and 5 have custom AI routines to control their own assault logic and resources.

Combat Flow and Awareness: In order to prevent loops, the AI maps out blocked paths after collisions. Additionally, an attack cooldown system and a short "freeze" time that renders the bot temporarily immobile following a successful strike are used to control combat pacing.

Attack system

All the characters have some specific mechanics.

Character 1 (Fire)

The Fire character is meticulously designed to be the overarching All-Rounder of the roster. Functioning as a traditional, fundamental fighting game combatant, he draws heavy inspiration from classic archetypes—most notably Ryu from the Street Fighter franchise. This makes him an excellent baseline character with a balanced, highly accessible toolkit equipped with a standard projectile to control the screen.

- **Animation Rigging:** In terms of visual execution, his animation setup is deliberately basic and straightforward. Every single one of his attacks is tied to exactly one dedicated animation, removing any complex branching or multi-stage attack strings from his base kit.
- **Combat Philosophy:** His primary win condition revolves around sheer, unadulterated damage output. He does not rely on complex traps or movement tricks; instead, he dictates the pace of the match by punishing the opponent through a highly synergistic elemental status engine.
- **Skill & Status Synergy:** The Fire character's offensive pressure is entirely built upon the interaction between his abilities. Both his S1 and S3 attacks are designed to directly inflict the Burn status on the enemy. Conversely, his S2 is used to apply Oil. The core mechanic requires the player to effectively weave these skills together—oiling the opponent to prime them, and then igniting them with his burning attacks to inflict massive, compounded damage.

Character 2 (Water)

This medium-speed crowd-control fighter features a unique 1.5 health bar (3 blue and 3 green chunks) and tailored resistances, taking 1.5x damage from blunt attacks, 1x from pierce, and 0.5x from slash. Visually managed via hitboxes (pink damage zones) and hurtboxes (green body zones), this character remains vulnerable during all actions except for the S3-2 animation. Its kit focuses on single-target disruption and displacement: S1 is a long-range, double-hitting

pierce attack that applies the WET status and plays "effect S1" exclusively on the first target hit. S2 serves as a short-range, blunt AoE that hits once and pushes all enemies away. Meanwhile, S3 functions as a tactical single-target blunt skill; if the initial tongue grab (S3-1) connects, it immediately triggers the invulnerable chew-and-spit sequence (S3-2), trapping the enemy in a forward-drifting bubble ("effect S3") that inflicts both WET and DISABLED statuses for 5 seconds.

Character 3 (Ghost)

Standing in stark contrast to the grounded nature of the Fire elemental, the Ghost firmly embodies the Glass Cannon archetype. He completely sacrifices durability and defensive viability in exchange for blistering speed, immense offensive power, and high technical versatility.

- The Fragility Factor: Because he is exceptionally frail, the Ghost cannot afford to trade blows or make careless mistakes in the neutral game. To compensate for this severe lack of survivability, his entire playstyle is highly adaptive, giving the player a wide variety of tactical options depending on the immediate situation.
- The "Weakened" Catalyst: The fundamental cornerstone of the Ghost's mechanical design is his ability to inflict the Weakened status on his foes. This status is not merely a stat reduction; it serves as a gateway mechanic that opens up his devastating secondary options.
- Situational Domination: Once the opponent is successfully weakened, the Ghost can completely overwhelm them by applying a myriad of crippling conditions tailored to the exact moment in combat:
 - Bleeding: Applied to slowly drain the enemy's health pool during hit-and-run scenarios.
 - Paralysis: Utilized as a hard lockdown tool, freezing the opponent to safely reposition or to prevent any form of retaliation.
 - Opening: This is undeniably his most lethal status effect. When inflicted, it creates a massive vulnerability, forcing the enemy to take x1.5 damage from absolutely all of the Ghost's subsequent attacks. This allows him to rapidly melt the opponent's health bar in a matter of seconds, provided he can capitalize on the window of opportunity.

Character 4 (Electric)

This high-utility combatant revolves entirely around managing a CHARGE mechanic, tracked by a HUD battery icon holding up to four charges that regenerate passively by +1 every 15 seconds. The character's combo routing and skill effectiveness change dynamically based on resource generation and consumption. S1 is a dash attack that grabs the target on contact, initiating a conditional sequence where a successful hit routes from S1-1 through S1-2 to S1-3 (granting +2 charges), whereas a miss skips directly to S1-3 without generating resources. S2 adapts to current battery levels: if under 3 charges, it executes S2-1, which deals no damage but generates +1 charge; if at 3 or more charges, it activates S2-2, consuming 2 charges to inflict a STUN status. S3 serves as the ultimate resource spender, consuming all remaining charges to deal damage and inflict a STUN status that scales in duration based on the exact number of charges spent.

Character 5 (Food)

This projectile-based zoner utilizes a dynamic ammo management HUD displaying two separate rows of four cans (Water and Soda) to dictate its offensive capabilities. The character's primary offensive options rely heavily on inventory count, altering its moveset entirely when running empty. Using the first skill consumes 1 Water can to fire Projectile 1, inflicting the WET status, while the second skill consumes 1 Soda can to fire Projectile 2, inflicting the OILED status. If the player attempts to fire either skill without the corresponding ammo, the attack automatically downgrades to S2, a melee panic-option that inflicts a PUSH status to create distance. Finally, S3 acts as a pure utility reload mechanism that deals no damage but completely restores both the water and soda can reserves back to their maximum capacity of 4.

Menu

Session's system

The session system was designed around a deliberate architectural choice: rather than storing session data in a database, sessions are held directly in memory as a variable within the server's object. This approach was made possible by the fact that the server runs continuously, staying active at all times regardless of whether any clients are connected. As a result, any session that has been created persists for as long as the server remains running — players can close and relaunch the client entirely, and when they reconnect, all existing sessions will still be visible and joinable in the session browser exactly as they were left. This keeps the system lightweight and avoids the overhead of database read and write operations for what is inherently temporary, session-based data.

When creating a new session, the host is prompted to define two parameters that determine how the session will be structured:

- Session title — a name chosen by the host that identifies the session in the browser, making it easy for other players to find and join.
- Number of bots — the number of AI-controlled players that will participate in the match.

The total number of human players in a session is derived automatically from the number of bots, following a fixed formula: the combined total of human players and bots must always equal four. This means that if a session is created with two bots, two human players are required for the game to start — the host and one additional player who joins through the session browser. A session with two bots and only one human player connected will remain in a waiting state until the second human player joins, at which point the game launches automatically. This design ensures that every match always has exactly four participants, maintaining a consistent and balanced game experience regardless of how many human players are involved.

Maps selection

Before a game session begins, players are taken through a selection process that determines both the map and the characters for the upcoming match. The map selection comes first. Any bots participating in the session each independently choose a map at random from the two available options — the Forest and the Desert. The player then makes their own choice, selecting whichever of the two maps they wish to play on. The bots' selections are revealed after the player has made their choice, ensuring that the player's decision is made freely without being influenced by what the bots have picked.

Character selection

Once the map has been decided, the session moves into the character selection phase. As with the map, bots each randomly select one of three characters assigned to them, with their choices remaining hidden until after the player has confirmed their own selection. The player, on the other hand, is presented with a visual interface through which they can browse the available characters and make their choice. This interface was designed to give the player a clear and accessible overview of the roster before committing to a selection. There are no time constraints on either the map or character selection screens, giving the player as much time as they need to make their decision before the match begins.

Gameplay

The gameplay mechanics are driven by displaying sequential frames directly from Aegon's custom spritesheets. When it comes to collisions and hitboxes, the system cuts through the noise by using simple geometric rectangles. By detecting

whether a player or bot occupies these specific coordinates, the game manages physics seamlessly. This rectangular collision system ultimately became the backbone of the project's technical design.

Art

The visual identity of Blitzkrieg is the result of a sustained and detailed artistic effort that touches every aspect of the game the player sees and interacts with. The art production can be divided into three distinct categories: the characters, the menus, and the maps. Aegon, serving as Lead Artist and Art Director, was responsible for the overwhelming majority of this work, producing all character sprites, most of the menu screens, and two of the three playable maps entirely from scratch. Adam, the team's secondary artist, complemented this work by contributing to a number of menu assets and designing one of the selectable maps, ensuring a consistent visual standard across the project.

Characters

Character art represents the most technically demanding component of the visual production. Each playable character required a complete set of assets to function correctly within the game: a player icon used in the HUD and selection screens, a death sprite, and a full suite of animations covering every possible in-game action — idle stance, movement, each of the character's three skills, and any associated projectiles or visual effects tied to those skills.

The animations for certain attacks are deliberately divided across multiple sprite sheets, each representing a distinct phase of a skill's execution. This approach allows the game to respond dynamically to what happens during an attack, rather than playing a single fixed animation regardless of outcome. The Ghost character illustrates this well: the first skill is a counter, and its animation is split into three phases. The first sheet, S1-1, covers the active counter window — the period during which the character is primed to intercept an incoming attack. If the character is successfully hit during this window, the animation transitions immediately to S1-2, the counterattack sequence. If no hit is registered, the animation instead flows into S1-3, a brief recovery period during which the character is left temporarily vulnerable. This branching animation structure gives

the skill genuine tactical depth, with the visual feedback directly communicating the outcome of each phase to both the player and their opponent.

This level of animation detail was applied consistently across all five characters, with each skill and effect animated individually to ensure clarity and visual responsiveness in gameplay. The twelve principles of animation were applied throughout the character work, giving attacks a sense of weight, movement a sense of fluidity, and each character a distinct visual personality that reinforces their role in the roster.

Menus

The menu artwork encompasses all of the visual interfaces the player encounters outside of active gameplay. This includes the main menu, the options screen, the session browser, the character selection screen, and the map selection interface, among others. Each screen was designed not only to be functional but to maintain the unified visual identity established for the project — consistent use of the game's colour palette, typography, and iconography ensuring that the transition between screens feels seamless and intentional. The menu art was produced collaboratively between Aegon and Adam, with Adam contributing a number of assets and interface elements that were integrated into the broader visual framework Aegon had established.

Maps

The maps are the environments in which all combat takes place, and each was designed to provide a visually distinct backdrop that reinforces the atmosphere of the game while keeping the play area clear and readable. Blitzkrieg features three maps in total: the Forest and the Desert, both designed by Aegon, and the Crystal Cave, designed by Adam. Each map was built to offer a different visual and spatial experience — from the dense, enclosed canopy of the Forest, to the open, sun-drenched expanse of the Desert, to the luminous, otherworldly depths of the Crystal Cave — while sharing the same underlying arena structure that ensures gameplay consistency across all three environments.

3- Issues encountered during the development.

Period	Issue
October	The first main issue encountered was for the organization of the project → We chose discord as a communication tool for all the communication's parts. And we chose git as a project management tool, for the programming part.
January	The second issue was in order to find a host: a LAN server system is great, but an online server is better. We find Katabump after long hours of research, which is a useful solution for transforming the server into an online server.
March	We observed a delay in the designer's work due to his health issues. We decided in consequence to make less designs in order to take care of the designer. This consequence hasn't removed any of the functionalities of the project, even though there is not as much design as we wanted to.
May / first half.	We already implemented the main functionalities of the project, but several critiques display bugs

	spread into our project. We decided to handle them quickly for a better user interface.
May / Second half	The AIs' development was a bit complicated. So, we proceeded step by step: 1. The first step was to code the actions. In order to do so, we randomly took action after define each action (follow the player, flee the player, attack, or a random action like go up, go down etc. Or just do nothing). 2. After programming all the action, the behavior is an important part to avoid an ia whose spamming random actions without any "intelligence". To do so we have to code the "brain" of the AI (his reflection / behavior). Depending on the situation, the bot pick an action with possibilities to append. For instance, when the bot has less than 20% of health, the bot will prefer flee to attack or follow the player. 3. After that, we just have to handle the player condition part. Meaning that a bot, as a player, has to handle the cooldowns, the collisions, status etc. 4. Then, some specific behavior will be implemented, like an anti-spam system, or a rage behavior.

4- The website

The Blitzkrieg project's website has been developed in collaboration by Yloan, David, and Adam, with Adam creating the graphics and visual design and David and Yloan coding the implementation of the website using HTML, CSS, and JavaScript. The site will serve as a way to show the game externally (to those who would not be able to see it first-hand) in an ordered and clear way. There are multiple sections to the site, including general information about the game, an overview of the three core gameplay pillars (the status and synergy system, team building, and character diversity), an overview of the gameplay controls and mechanics, and an explanation of how the session and room system works for multiplayer. There is also an external resources section that lists libraries, tools, and musical references that were used throughout development.

The design of the site was intentionally designed to match the game's in-game user interface with the use of the same green and cyan color palette that was used throughout all of the menus within the game; further, this reinforces the consistency of aesthetic across all aspects of the Blitzkrieg project.

5- Each group member's implication

Yloan

Throughout this project, I took on a dual role as both the team's organiser and its lead programmer, and in practice, these two responsibilities defined the majority of the work I contributed across the entire development cycle.

On the organisational side, I was the one who set the team's infrastructure in place from the very beginning. I created the Discord server, configured the bots,

and established the communication channels that the team relied on for the duration of the project. I also set up the GitHub repository, designed its architecture, and put in place the folder structure and conventions that kept the codebase navigable as it grew. Beyond the tools, I organised and led our regular meetings, ensuring the team stayed aligned on priorities and that progress was being made consistently week to week.

On the programming side, my contribution was the most substantial of the team. I was the primary developer responsible for building and implementing the majority of the game's core systems. This included the entire server and room system, the session browser that allows players to create and join game sessions, which I built from the ground up entirely on my own. I also developed the AI system in full, another major technical undertaking that formed a critical part of the game's functionality. In addition to these systems, I contributed to the menu interface alongside Quentin, who led the character selection portion; my own focus within the menu was the session browser, for which I handled the complete implementation independently (interface made by Adam). Moreover, the whole gameplay has been made by Quentin and me.

In short, while the project was a team effort and I was grateful for the support of my teammates, the bulk of the technical architecture, core systems, and organisational groundwork was work I took on and delivered myself.

David

Throughout this project, my main role was that of music composer and audio implementer, though my contributions extended beyond those areas as well. On the music side, my work can be split into two parts: composition and implementation.

In terms of composition, I created one original music piece using BandLab, which serves as the game's menu music. As mentioned previously, the initial plan was to use and slightly modify existing songs for the remaining tracks, but this had to be reconsidered once we realised that the songs we had in mind were not cleared for use. I therefore had to find royalty-free, unlicensed music pieces that would still fit the tone and aesthetic of the game. After searching for suitable

tracks, I found two pieces that matched the game's style well. The first is "Retro Egyptian Theme" by Howling Hound Music, a royalty-free track that fits the game's old-school pixelated atmosphere and adds a strong sense of identity to one of the in-game maps. The second piece has similarly been sourced and is fully unlicensed, making both tracks safe to include in the final product. Both songs were edited as needed, trimming their intros and outros so that they could loop seamlessly in-game without any audible breaks.

On the implementation side, I was responsible for integrating all three music pieces into the game. This involved creating a MusicPlayer class in Python using Pygame, which handles playing, looping, pausing, unpausing, and stopping music depending on what is happening in the game. The class takes in music files in MP3 format and automatically switches between the menu music and the in-game battle music at the appropriate moment. Getting this to work required me to switch from Ubuntu on a virtual machine to VSCode on Windows, as Ubuntu did not allow me to properly test the Pygame audio output. This was a significant change to my development setup, but it was necessary to ensure the music worked correctly.

Beyond my audio responsibilities, I also contributed to other areas of the project. I helped with the implementation of the characters, assisting my teammates in getting the character systems working in the game. More generally, I made myself available throughout the project to support wherever help was needed, whether that meant giving feedback, helping debug issues, or contributing to parts of the project outside my primary role. This kind of cross-team involvement was something I found valuable, as it gave me a broader understanding of how all the pieces of the game fit together.

Adam

During my participation in this project, I created three main content types: visual designs, maps, and web content. While these design projects are not related to each other, they are all intended to come together and have cohesive, polished, and functional design for the entire project.

The first thing I did in terms of design was to create the visual language for the interfaces in the game. I created the visual language for all of the screens that would be shown to players who are interacting with the game prior to playing, such as the main menu screen, the page where a player can browse sessions, the session creation page, and the map selection page. I created the visuals used in these screen designs using multiple programs: Adobe Photoshop for layered designs, Adobe Illustrator for graphics designed as vector images (to allow for resizing and high-resolution printing later), and Canva for quick prototypes of the designs.

The design system I created has been carefully constructed using a green and cyan color scheme with the intention of being the same throughout each screen so that they can have a singular visual identity that carries through from the first screen that a player sees to all of the screens included in the web. All of the assets were intended to be modular in such a way that they could be used over and over again in different locations throughout the various menus by developers without requiring a lot of rework on their part, thus making it easy to integrate the assets into the menus they were creating. Also, any interface would be revised many times based upon feedback from the entire development team, notably from Quentin, who had the responsibility to implement the menu system for the game (with input from other team members as determined by how much input they provided); and Yloan, whose structure for integrating into the session browser provided a great deal of insight for creating the visual layout for the information on the various interfaces.

On the map side, I created the crystal map, which is one of the three playable areas in the game. The crystal map was a different kind of creative challenge than the interface work I had done before; where UI design focuses on clarity and navigation, with map design, you're trying to create an atmosphere and a place out of the pixel art format and visual direction of the game. I worked with Aegon closely to make sure that the aesthetic of the map was in line with the general art direction of the game, with specific attention to things like colour palette, tile density, and making sure that each layer of the map, the background, and foreground function independently as separate assets.

On the web side of things, I was responsible for the visual design of the project website, which is hosted on Vercel, including all CSS. I structured how the website would look (from the layout to how styles were applied) so that everything on the website would have the same visual identity as the game itself. People visiting the site would see the same colour palette, tone, and typeface when they came to visit. David created the underlying HTML structure and implementation of the website, and we worked very closely together during the entire process to ensure that the design looked the same in the browser as it did in person.

Overall, this project gave me a much better understanding of how design is part of a developer's workflow; it isn't just a final touch after all development is done, but also a constraint and a guide for the development team in making technical decisions from day one. The best moments I had were when my designs made it easier for developers to build something rather than harder.

QUENTIN

Throughout this project, my contributions were centred around the game's user interface and the character systems, two areas that proved to be among the most technically involved parts of the project.

On the interface side, I was responsible for the design and implementation of the game's menu system. This covered the main menu that players encounter when launching the game, as well as the character selection screen, which I built and iterated on across multiple versions as the team refined what the selection flow should look like. I also implemented the in-game pause menu, which required careful integration into the existing game loop to ensure it could overlay the frozen game frame correctly without disrupting the rendering pipeline. Getting this right involved identifying and resolving a rendering bug introduced during the process, where an intermediate surface I had added caused a persistent black square to appear under the HUD — tracking down the root cause and reverting only the problematic parts of the pipeline while preserving the pause menu

behaviour was a good example of the kind of careful debugging work the project demanded.

On the character side, I contributed to the implementation of the playable characters within the game, working to get their animations, states, and in-game behaviour functioning correctly alongside my teammates.

Beyond my own primary areas, I worked closely with Yloan throughout the project to fix various bugs that emerged as the different systems were integrated together. This collaborative debugging was an important part of keeping the project moving forward, as many of the issues only surfaced once multiple systems were running at the same time.

More broadly, working on a project of this scale gave me a much clearer sense of how quickly complexity accumulates when several systems need to interact — and how much of the real work in a game project lies not in building individual features, but in making sure they hold together correctly as a whole.

AEGON

From the very first stages of the project, my role extended far beyond that of a visual contributor. As Lead Designer and Art Director, I was responsible for the creative foundation upon which the entire project was built, from the original concept that sparked the game's existence, to every sprite, animation, menu screen, and sound effect that made it feel alive.

The idea for Blitzkrieg originated with me. Drawing on a passion for mechanical creature design that dated back to a drawing competition I had participated in during my childhood, I brought the initial concept to the team at the start of the year. The vision of a fast-paced, character-driven pixel art fighting game with a roster of uniquely designed combatants was one I had carried for some time, and this project gave it a concrete form. Beyond the visual concept, I was also

responsible for designing the game's core mechanical identity, specifically the status effect system and the synergy interactions between effects that sit at the heart of Blitzkrieg's gameplay. The idea that status effects could interact with one another in meaningful ways, oil amplifying fire damage, water enhancing push distance, and so on, was a concept I developed and brought to the team, and it became one of the most distinctive and praised aspects of the final product.

On the production side, most of the visual content in the game was created by me, entirely from scratch. This includes all five playable characters and their full animation sets. I designed and built two of the three maps, the Forest and the Desert, as well as most of the menu screens, HUD element, and interface component that the player encounters throughout the game. Each visual effect tied to a skill or status interaction was also animated individually, giving the game a level of visual feedback that made it readable and engaging in motion. In addition to this, I made all of the sound effects used in the game, ensuring that every action, from a character being hit to a skill being activated, was paired with an appropriate audio cue that enhanced the gameplay experience without requiring the player to look at the screen.

It is important to acknowledge that the scope of what was ultimately delivered does not fully reflect the original ambitions set at the start of the project. The initial plan called for a roster of eighteen playable characters and a larger set of status effects. Due to a personal health condition that affected my availability and capacity over a significant portion of the development period, the team made the collective decision to reduce the character roster to five and to scale back the number of status effects accordingly. This was the right decision and one made with the full support of the team, but it is worth stating clearly that the reduction in scope was a direct consequence of circumstances beyond my control, and not a reflection of the quality or commitment of the work that was delivered. The five characters that were completed, each fully animated, visually distinct, and mechanically unique, represent a substantial body of work, and every element of the game's visual and creative identity was shaped by the contribution I made throughout the year.

4- Communication and organization

Effective communication and clear organisation are two of the pillars of any successful team project, and our FPS game development was no exception. With a team of five people working across different areas, programming, game design, art, and audio, establishing reliable communication channels and structured working habits was a priority from the very start. This section covers how our team organised itself, how we made use of Discord as our primary communication and coordination platform, and how our weekly meetings helped us stay aligned on goals and progress.

Team Organisation

Before diving into production, the team took time to define roles and responsibilities for each member. This initial step was essential to avoid overlap, reduce confusion, and make sure every area of the project had a clear owner. While collaboration across disciplines was encouraged and happened naturally throughout the project, each person held a primary responsibility that guided their contributions.

The team was divided into the following areas of responsibility:

- Game Design — responsible for defining the gameplay mechanics, player experience, and character/map designs
- Programming — in charge of implementing the core systems of the game, including player controls, AI behaviour, attack mechanics, and overall game logic using the chosen game engine.
- Audio & Sound Design — in charge of sourcing, creating, and integrating sound effects and background music to enhance the game's atmosphere.
- Project Management & QA — overseeing timelines, task tracking, and ensuring the quality of deliverables through regular testing and feedback cycles.

This distribution allowed the team to work in parallel on different aspects of the project without stepping on each other's work. It also made it easier to identify who to turn to whenever a specific question or problem arose, which noticeably sped up decision-making.

Despite having well-defined individual roles, several tasks were treated as shared responsibilities across the team. Playtesting, for instance, was something every member participated in, since having multiple perspectives on gameplay feel and difficulty was invaluable. Similarly, decisions about the game's overall direction, such as significant design pivots or feature cuts, were made collectively during meetings rather than unilaterally.

This balance between individual ownership and collective decision-making created a healthy working dynamic: each member had the autonomy to make progress on their tasks independently, while still feeling invested in the project as a whole.

Communication Tools - Discord

Discord was chosen as the team's main communication platform. Already widely used in the gaming community, it offered a familiar, feature-rich environment that suited our project both practically and thematically. It allowed us to centralise all communication, file sharing, and meeting activity in a single place, reducing the need to juggle multiple tools.

To keep communication structured and easy to navigate, the team set up a dedicated Discord server with channels organised by topic. Rather than routing all discussions through a single general channel, this approach allowed each area of the project to have its own dedicated space, making it much easier to search through past conversations and keep discussions focused.

The server was organised into several categories:

- General — a #general channel for casual team conversation, non-project chat, and day-to-day messages, as well as a #announcements channel where important updates, deadlines, or decisions were posted for everyone to see.
- Development — dedicated text channels for each discipline: #programming, #game-design, #art-and-assets, and #audio. These channels served as spaces for sharing work-in-progress, asking domain-specific questions, and posting updates.
- Resources & Files — a #shared-files channel for posting important documents, reference images, and downloadable assets, as well as a #links channel for bookmarking useful tutorials, tools, and external references.
- Meetings — a #meeting-notes channel where summaries of every weekly meeting were posted, allowing members who missed a session to catch up and ensuring a written record of all decisions made.
- Voice Channels — dedicated voice rooms for team meetings and informal co-working sessions, making it easy to jump into a call whenever real-time communication was needed.

Beyond formal meetings, Discord served as the backbone of our day-to-day communication. Members regularly posted updates on what they were working on, shared screenshots or short videos of work-in-progress features, and flagged any blockers they were facing. This informal but consistent flow of updates meant that the whole team had a reasonably clear picture of progress at any given time, even outside of meeting hours.

The use of Discord's reaction features also helped streamline lightweight feedback: a quick thumbs-up emoji could signal agreement with a proposal, while a question mark reaction would prompt further discussion. This kept communication efficient and reduced the pressure to respond immediately to every message with a full written reply.

To enhance the server's functionality, the team integrated a number of bots and automation tools. These additions helped reduce administrative overhead and keep everyone informed of project milestones without requiring manual updates.

- **Task Tracking** — a bot was configured to interface with our task management system, allowing members to create, update, and check on tasks directly from Discord without needing to switch to a separate platform. This kept the team's to-do list visible and up to date within the communication hub itself.
- **Reminders** — automated reminders were set up to notify the team ahead of weekly meetings and approaching deadlines, helping ensure that no one missed an important session or submission window.

These integrations turned Discord from a simple chat platform into a genuine project hub, a single place where communication, task management, and version tracking all came together.

Regular Meetings

In addition to asynchronous communication through Discord, the team committed to holding a structured weekly meeting. These sessions played a crucial role in maintaining momentum, surfacing issues early, and ensuring that all members remained aligned on the project's direction and priorities.

Meetings were held once per week, typically at a fixed day and time agreed upon by the team at the start of the project. Holding meetings at a consistent schedule made it easier for everyone to plan their week and ensured a reliable rhythm of communication throughout the development cycle.

All meetings took place via Discord's voice channels. This was a natural choice given that Discord was already the team's main communication platform, and using voice channels eliminated the need for an additional tool. Screen sharing

was used frequently during meetings, particularly when a member wanted to demonstrate a new feature, walk through a level design, or share reference material.

Each session typically lasted between 45 minutes and 1.5 hours, depending on the volume of topics to cover.

Meetings generally followed a consistent structure, which evolved slightly over the course of the project but retained the same core elements:

- Round of updates — each member briefly shared what they had worked on since the last meeting, what was completed, and what was still in progress. This gave the whole team a quick, shared understanding of the project's current state.
- Blockers and issues — members flagged anything that was preventing them from making progress, whether a technical issue, a design question requiring group input, or a dependency on another team member's work. This part of the meeting was particularly valuable for unblocking individuals quickly.
- Decisions and discussion — key decisions that could not be resolved asynchronously were brought to the meeting for collective deliberation. This might include changes to gameplay mechanics, art direction choices, or adjustments to the development schedule.
- Goals for the next week — the meeting concluded by defining clear, achievable objectives for each member to work on before the following session. These goals were posted in the #meeting-notes channel after the call, serving as both a reminder and an accountability mechanism.
- Others — a brief open floor for any topics not covered by the agenda, allowing members to raise anything on their minds before the meeting wraps up.

Follow-up tasks identified during meetings were added to the team's task tracker, ensuring that action items did not get lost in the flow of conversation. Each task

was assigned to a specific person with a target completion date, creating clear accountability without the need for heavy project management overhead.

As the project evolved, so did the rhythm and content of the meetings. During the early planning and pre-production phases, meetings tended to be more exploratory and discussion-heavy, as the team worked through design decisions and established the project's foundations. In the production phase, sessions became more task-oriented, focusing on reviewing progress against milestones and identifying risks early.

Towards the end of the project, as the final build approached, the team occasionally supplemented weekly meetings with shorter, ad hoc check-ins via voice or text to address time-sensitive issues or review builds before submission. This flexibility, maintaining a consistent weekly rhythm while leaving room for more frequent contact when needed, was one of the strengths of the team's communication approach.

Summary

The combination of clear role distribution, a well-structured Discord server, and consistent weekly meetings gave the team a solid organisational foundation throughout the project. Communication was kept transparent and accessible, with information centralised in a single platform that every member was comfortable using. Regular meetings ensured that progress was visible, blockers were addressed promptly, and goals remained clear from week to week.

While no team workflow is without its challenges, and this project was no exception, the structures put in place from the start meant that communication rarely became a source of friction. If anything, the experience reinforced the importance of investing time early on in setting up good communication habits, as the returns in productivity and team cohesion throughout the project were significant.

7- IF we can restart, what will we change

Looking back on the project, there are several things we would approach differently if we were to start over.

The first concerns the scope of the project. In hindsight, we set overly ambitious goals from the beginning, planning for a large number of characters, maps, and features. If we were to redo the project, we would scale this back significantly and instead focus our energy on delivering a smaller but more polished experience — prioritising smooth animations, fluid transitions, and a refined overall feel over sheer quantity of content. A product with fewer elements but higher quality would have been both more achievable and more satisfying to deliver.

Closely linked to this is the issue of planning. Looking back, we would have benefited greatly from investing more time at the very start of the project in thorough, detailed planning — clearly defining the scope, setting realistic milestones, and anticipating potential roadblocks before they arose. Alongside this, we would have been stricter with our deadlines throughout the project. While the team worked well together, there were moments where flexibility around due dates allowed delays to accumulate. In hindsight, holding ourselves to firmer deadlines from the start would have kept the project on track and reduced the pressure felt in the later stages of development.

The third area is code organisation. One of the most important lessons this project taught us is that the architecture of a codebase needs to be thought through thoroughly before a single line is written. Without that foundation, the code quickly becomes difficult to navigate and maintain — something we experienced firsthand, particularly in the menu section of the project, which grew increasingly disorganised over time. If we started again, we would invest real time upfront in planning the structure of the code, enforcing clear documentation standards across the whole team, and requiring every member to explain and comment any code they added when pushing updates. This would have prevented a great deal of confusion down the line and made collaboration significantly smoother. On a related note, we would also adopt the use of branches in our version control workflow from day one, rather than introducing them later in the process.

Finally, the project was also affected by a difficult personal situation involving one of our team members, which had an impact on their availability and output over a period of time. As a team, we responded by reducing the workload and adjusting our expectations for that period to support them. While this was the right decision, it did result in some planned designs not being completed. If we were to redo the project, we would build more contingency time into our planning from the start, so that unforeseen personal circumstances could be absorbed without affecting the overall timeline as significantly.

8- Conclusion (what was learnt by the end of the project)

This project represented a significant milestone for every member of the team, not only in terms of the final product delivered, but in the breadth of skills and experience gained throughout the development process. Building a fully functional online multiplayer pixel art fighting game from scratch, in Python, over the course of a single school year, was an ambitious undertaking, and one that pushed each team member well beyond the boundaries of what they had previously been capable of.

For some members of the team, the project served as a first real encounter with game development altogether. Concepts such as game loops, sprite animation, collision detection, and real-time multiplayer networking were entirely new territory, and mastering them required considerable self-directed learning alongside the collaborative work of the team. For others, the primary growth came from deepening existing technical skills, learning to write cleaner, more structured code, working with version control in a team environment, or producing professional-quality assets and audio under real project constraints. Across the board, the project was as much an education in the craft of software development as it was in any specific technology.

Beyond the technical, the project also provided invaluable experience in teamwork and project management. Meeting deadlines, coordinating across

different areas of responsibility, supporting teammates through difficulties, and making collective decisions under pressure are skills that no individual assignment can fully replicate. The team learned, sometimes the hard way, the importance of clear communication, thorough upfront planning, and disciplined adherence to timelines.

In reflecting on the project's management, the MoSCoW method proved to be a particularly useful framework for prioritisation. By categorising features and tasks into Must-haves, Should-haves, Could-haves, and Won't-haves, the team was able to maintain focus on the core deliverables even as the scope of the project evolved. This approach helped prevent the project from becoming overwhelmed by ambition, and ensured that the most critical elements of the game were completed and functional even when time or resources became constrained.

Ultimately, Blitzkrieg stands as a testament to what a small, motivated team can produce when it commits fully to a shared vision. The project was far from perfect, and the team has identified clearly what it would do differently given the chance to start again. But those lessons are themselves part of the value of the experience. The technical knowledge, the collaborative habits, and the resilience built over the course of this project will serve every member of the team well beyond the walls of this school year.

Installation and Deletion

In order to install the project, go to the following website:
<https://website-first-defense-t5g6.vercel.app/> .

Then go to the section “Installer & Uninstaller”, choose the installer (mac or windows, linux can deal with the mac installation).

Then unzip the .zip installed, go to the terminals and launch the following command depending on the operating system:

Mac:

- `bash install_mac.sh`
- or `zsh install_mac.sh`

Windows:

- `install_windows.bat`
- or just double click on it in the file explorer

After install the project, at his root, execute the following commands depending on your operating system:

If you are on windows:

make setup-win

make run-win

If you are on linux / macOS:

make setup-mac

make run-mac

These commands will create a .venv, install the requirements and run the project. But windows is different from macOS and Linux, so you **must** handle these operating system cases.

For uninstall the project, you can go to the website again and download the uninstall depending on your operating system and repeat the same operation as before, which are :

Mac:

- `bash uninstall_mac.sh`
- or `zsh uninstall_mac.sh`

Windows:

- `uninstall_windows.bat`
- or just double click on it in the file explorer

Controls

Use the arrow keys to move your character across the battlefield.

Press the **A**, **S**, and **D** keys to activate the three abilities of the currently selected character.

Press **E + Q** to switch between Support Character 1 and the backup character.

Press **E + W** to switch between Support Character 2 and the backup character.